DAY 12

Forgot Password System

Time-limited reset links using itsdangerous, email service and bcrypt password update

token

email link

expiry

new hash

Trainer

Madhu Parvathaneni, CTO - Codegnan
madhu@codegnan.com - 8121289993

Where Day 12 fits in SNS

We already have registration, OTP verification, login and sessions. Today we recover account access safely.

Day 9: User registration stored verified users in MySQL
Day 10: OTP verified ownership of the email inbox
Day 11: Login created a session for verified users
Day 12: Forgot password lets a user reset access through a time-limited email link

Today’s outcome

Students implement /forgot-password and /reset-password/<token>, validate token expiry and update password_hash safely.

Key safety idea

Do not reveal whether an email exists. Always show a generic response after a reset request.

Learning outcomes

By the end of class students should be able to explain and implement the reset flow.

Concepts

Tokenization, signed tokens, timed serializer, salts, expiry and generic responses.

Implementation

Generate a reset token, build an email link, validate the token and hash a new password.

Security

Avoid account enumeration, protect secrets, handle expired/invalid links and never store raw passwords.

Evidence

Show success, invalid token, expired token, and login using the new password.

The Day 12 core flow

Students should memorize this before coding.

1. Forgot request
2. Generic response
3. Signed token
4. Email link
5. Expiry validation
6. New hash

Important: the reset link proves access to the mailbox, not knowledge of the old password.

Why not just send a new password?

Because password reset must protect identity, secrets and auditability.

Bad idea

Emailing a temporary password teaches insecure habits and creates another credential to protect.

Better approach

Send a short-lived link that allows the user to set a new password once.

Best project practice

The link contains a signed token. The app verifies integrity and expiry before updating password_hash.

Only hash is stored in users.password_hash
Reset link expires after configured max_age
Old password is never emailed, displayed or logged

Route map for Day 12

Keep the route design small and easy to test.

GET	**/forgot-password**	Show email form
POST	**/forgot-password**	Accept email, send generic response
GET	**/reset-password/<token>**	Validate token, show new password form
POST	**/reset-password/<token>**	Validate token again, hash and update password

What is tokenization?

A token is a compact signed proof carried inside the reset URL.

Payload

Minimal data such as user_id. Do not store password, email secrets or full user profiles.

Signature

Proves the application created the token and detects tampering.

Timestamp

Allows loads() to reject tokens older than max_age.

```
serializer = URLSafeTimedSerializer(app.config['SECRET_KEY'])

token = serializer.dumps(
    {'user_id': user['id']},
    salt='password-reset'
)
```

itsdangerous in one sentence

It signs data so the app can later check that the data was not modified.

Not encryption

Users may see encoded payload parts
Do not put sensitive values inside the token

Signing

Tampering breaks verification
Salt separates password-reset tokens from other token types

Timed validation

max_age limits lifetime
SignatureExpired is a normal failure path

Trainer phrase

“Signed means protected from modification; encrypted means hidden. Today we use signed timed tokens.”

Generating a reset token

The token is created after a valid email is found, but the browser response should stay generic.

```python
from itsdangerous import URLSafeTimedSerializer

def generate_reset_token(user_id):
    serializer = URLSafeTimedSerializer(
        app.config['SECRET_KEY']
    )
    return serializer.dumps(
        {'user_id': user_id},
        salt='password-reset'
    )
```

Use the same SECRET_KEY for dumps and loads
Use the same salt on generation and validation
Keep payload minimal: user_id is enough
Send token by email link, not as a password

Building the reset URL

The email contains a clickable URL with the token in the path.

```python
reset_url = url_for(
    'reset_password',
    token=token,
    _external=True
)

send_reset_email(user['email'], reset_url)
```

Why _external=True?

It builds the full URL including scheme and host, which is required when the user clicks from email outside the current browser request.

Email body example

Click the link below to reset your SNS password. This link expires in 30 minutes.

Generic response prevents account enumeration

Do not let attackers test which emails are registered.

Avoid

Bad response

“No account exists for this email.”

Problem: attacker can test email lists.

Use

Good response

“If an account exists, a reset link has been sent.”

Benefit: same result for known and unknown emails.

Same page, same message, same timing style where possible.

Validating the token

GET and POST reset routes must both validate the token.

```python
from itsdangerous import SignatureExpired, BadSignature

def verify_reset_token(token, max_age=1800):
    serializer = URLSafeTimedSerializer(
        app.config['SECRET_KEY']
    )
    try:
        return serializer.loads(
            token,
            salt='password-reset',
            max_age=max_age
        )
    except SignatureExpired:
        return 'expired'
    except BadSignature:
        return None
```

SignatureExpired: token was real but too old
BadSignature: token is invalid or tampered
A valid payload still needs user lookup
Never update password before token validation

GET reset route

Valid token shows the new-password form. Invalid token shows safe recovery.

```python
@app.get('/reset-password/<token>')
def reset_password_form(token):
    payload = verify_reset_token(token)

    if payload == 'expired':
        return render_template(
            'reset_invalid.html',
            reason='expired'
        ), 400

    if not payload:
        return render_template(
            'reset_invalid.html',
            reason='invalid'
        ), 400

    return render_template(
        'reset_password.html',
        token=token
    )
```

Class demo

Open a valid link, then edit one character in the token and refresh. Students should predict the response.

Question

Why do we pass token back to the POST form?

POST reset route

After validation, hash the new password and update users.password_hash.

```python
@app.post('/reset-password/<token>')
def reset_password_submit(token):
    payload = verify_reset_token(token)
    if not isinstance(payload, dict):
        return render_template('reset_invalid.html'), 400

    password = request.form.get('password', '')
    confirm = request.form.get('confirm_password', '')
    if password != confirm or len(password) < 8:
        return render_template('reset_password.html',
token=token, error='Check password')

    password_hash =
bcrypt.generate_password_hash(password).decode()
    cursor.execute(
        'UPDATE users SET password_hash=%s WHERE id=%s',
        (password_hash, payload['user_id'])
    )
    connection.commit()
    return redirect(url_for('login'))
```

Validate token before reading user_id
Validate password and confirmation
Hash new password with bcrypt
Use parameterized UPDATE
Redirect to login after success

SQL update pattern

The update must target exactly one intended user.

```sql
UPDATE users
SET password_hash = %s
WHERE id = %s;
```

Never forget WHERE

Without WHERE, every user password may be changed. That is a high-impact data incident.

```
cursor.execute(
    '''
    UPDATE users
    SET password_hash = %s
    WHERE id = %s
    ''',
    (password_hash, user_id)
)
connection.commit()
```

Reset link failure paths

Failure paths are part of the feature, not optional extras.

Expired token

Catch SignatureExpired and show “link expired”. Offer a new forgot-password request.

Invalid token

Catch BadSignature or validation failure. Do not expose token details.

Replay after success

Design a way to invalidate reset state, or at minimum force users through fresh link generation.

Test: valid token + valid password
Test: tampered token
Test: expired token
Test: password mismatch
Test: old password should no longer work

Security checklist

Use this as the board checklist before the live lab.

- Generic response for reset request
- Use URLSafeTimedSerializer with stable SECRET_KEY
- Use same salt for dumps() and loads()
- Set max_age on token verification
- Do not place sensitive data in token payload
- Hash the new password before UPDATE
- Use parameterized SQL
- Do not log tokens or reset links in public logs

Production note

Add rate limiting, audit logging, CSRF protection, token invalidation and email deliverability monitoring.

Live coding sequence

Do not reveal all code at once. Build the system in small working slices.

1. Show forgot form
2. Handle POST with generic response
3. Find user by email internally
4. Generate token
5. Send email link
6. Validate token in GET
7. Update password in POST
8. Test login with new password

Guided lab: acceptance criteria

Students must demonstrate successful and unsuccessful cases.

Forgot-password page is reachable and styled
Unknown and known emails show the same generic response
Valid reset link opens the new password form
Expired/invalid/tampered links are rejected safely
New password is hashed and committed to MySQL
Login works with new password and fails with old password
README updated with routes and configuration
Commit message: Day 12: implement forgot password system

Evidence required

Screenshots or terminal evidence for one successful case and two unsuccessful cases.

Trainer check

Ask one student to explain the whole flow without code.

Debugging drills

Introduce one defect at a time and force students to reason by layer.

Symptom	Likely cause	Fix direction
Token invalid immediately	Secret key or salt mismatch	Use same SECRET_KEY and salt
Expired token crashes app	SignatureExpired not caught	Add exception handling
Password not updated	Missing commit or wrong WHERE	Check rowcount and commit
Reset link works repeatedly	No invalidation strategy	Add recovery flow discussion

Viva questions

Use as oral check or exit ticket.

Why do we show a generic response for forgot password?
What does URLSafeTimedSerializer do?
What is the difference between signing and encryption?
Why do dumps() and loads() need the same salt?
What happens when max_age is exceeded?
Why should the reset token be validated again on POST?
Why do we hash the new password before UPDATE?
What failure case did you test today?

Homework assignment

Submit a working Day-12 milestone before next class.

Implement forgot-password request and reset-password routes
Send reset email using the existing mail service
Use itsdangerous with max_age=1800 seconds
Handle expired and invalid token pages gracefully
Update password_hash with parameterized SQL
Capture evidence: success + expired/invalid/tampered case
Update README with routes, environment variables and test cases
Commit: Day 12: implement forgot password system

Submission

GitHub link + screenshots + README update + exact commit message.

Next class

Notes CRUD Part 1: authenticated add note and owned notes list.

Day 12 recap

Students should leave with a safe account recovery mental model.

Forgot password is an identity recovery flow through email
Tokens are signed and timed, not encrypted
Generic responses reduce account enumeration risk
Every reset link must be validated before showing or processing the form
New passwords must be hashed before UPDATE
Failure paths are expected: expired, invalid, tampered, replay

One-line memory

Forgot request → generic response → signed token → email link → expiry validation → new hash